

응용프로그램을 작성할 때 표준 라이브러리 함수 (예 : printf())를 호출하는 것이 시스템 호출 함수 (예 : write())를 호출하는 것보다 나은 방법인 이유를 설명하라.

1. 성능 최적화 (버퍼링): 시스템 호출은 사용자 모드에서 커널 모드로 전환되는 비용(오버헤드)이 매우 큼니다. printf는 데이터를 내부 버퍼에 모아두었다가 한 번에 전달하는 버퍼링 메커니즘을 사용하기 때문에, 매번 write를 호출할 때 발생하는 불필요한 모드 전환을 획기적으로 줄여 실행 속도를 높여줍니다.
2. 이식성 (Portability): 시스템 호출은 운영체제마다 인터페이스와 동작 방식이 다르지만, 표준 라이브러리는 ANSI C와 같은 공통 규격을 따릅니다. 따라서 표준 함수를 사용해 작성한 코드는 윈도우, 리눅스, 맥 등 다양한 OS 환경에서 소스 수정 없이 컴파일하여 그대로 실행할 수 있는 장점이 있습니다.
3. 사용 편의성 및 기능성: write는 단순히 메모리의 바이트 데이터를 전송하는 저수준 작업만 수행하는 반면, printf는 숫자, 문자열, 실수 등 다양한 데이터 타입을 사람이 읽기 좋은 형태로 변환해주는 포매팅 (Formatting) 기능을 기본으로 제공하여 개발 생산성을 높여줍니다.

2. 컴퓨터 시스템과 운영체제의 목표는 CPU의 활용률을 극대화하는데 있다. 인터럽트는 이 목적을 달성하게 하는데 중요한 역할을 한다. 인터럽트가 어떤 식으로 CPU 활용률을 높이는데 기여하는지 설명하라.

1. 비동기적 I/O 처리 (입출력 대기 시간의 활용)

I/O 장치(디스크, 키보드, 네트워크 등)는 CPU에 비해 속도가 매우 느립니다. 만약 인터럽트가 없다면, CPU는 입출력이 끝날 때까지 장치의 상태를 계속 확인하며 기다려야 합니다(폴링, Polling). 하지만 인터럽트 방식에서는 CPU가 I/O 명령을 던져놓고 즉시 **다른 작업(프로세스)**을 수행합니다. 이후 I/O 처리가 완료되면 장치가 인터럽트 신호를 보내 CPU를 호출하므로, CPU는 단 1초도 낭비하지 않고 계속 일할 수 있습니다.

2. 멀티프로그래밍과 시분할 시스템의 구현

인터럽트는 여러 프로그램을 동시에 실행하는 것처럼 보이게 하는 멀티프로그래밍의 핵심 동력입니다.

- 타이머 인터럽트: 운영체제는 타이머 인터럽트를 사용하여 특정 프로세스가 CPU를 독점하지 못하게 합니다. 일정 시간이 지나 인터럽트가 발생하면, CPU는 현재 작업을 멈추고 다음 프로세스로 전환(Context Switching)합니다. 이를 통해 CPU는 쉬지 않고 여러 작업을 번갈아 처리하며 전체 시스템의 처리량(Throughput)을 극대화합니다.

3. 예외 상황 및 즉각적인 이벤트 대응

프로그램 실행 중 발생하는 오류(Zero Division 등)나 하드웨어 이벤트가 발생했을 때, CPU가 이를 일일이 감시할 필요가 없습니다. 사건이 터졌을 때만 인터럽트가 CPU의 주의를 끌기 때문에, 평상시 CPU는 오직 연산 작업에만 집중할 수 있어 자원 활용 효율이 높아집니다.